



UNIVERSIDAD DE LAS FUERZAS ARMADAS- ESPE - MATRIZ

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN



ASIGNATURA

Metodología de la investigación

TEMA

Caja Blanca

INTEGRANTES

Jelen lucio, Jaime Cabezas, Bryan Cevallos

NIVEL-PARALELO - NRC

NRC : 30693

FECHA DE ENTREGA

03/07/2026

SANGOLQUI – ECUADOR

Requisito Funcional N.º REQ008: Chat en Tiempo Real por Documento

CÓDIGO FUENTE PARA Chat en Tiempo Real por Documento

```
async def enviar_mensaje(request: Request, documento_id: str, datos: MensajeChat,
usuario_actual: dict = Depends(obtener_usuario_actual)):

    documento = await request.app.state.db["documentos"].find_one({"_id":
ObjectId(documento_id)})

    if documento is None:

        raise HTTPException(status_code=404, detail="Documento no encontrado")

    nuevo_mensaje = {

        "remitente": usuario_actual["correo"],

        "mensaje": datos.mensaje,

        "fecha": datetime.now().isoformat(),

    }

    await request.app.state.db["documentos"].update_one(

        {"_id": ObjectId(documento_id)},

        {"$push": {"mensajes": nuevo_mensaje}}

    )

    # Persistir notificación y emitir WebSocket al receptor

    remitente = usuario_actual["correo"]

    titulo_doc = documento.get("titulo", "documento")
```

```
posibles_receptores = {documento.get("asignado_a"), documento.get("creado_por")}

for receptor in posibles_receptores:

    if receptor and receptor != remitente:

        ahora = datetime.now().isoformat()

        notif = {

            "usuario_receptor": receptor,

            "titulo": "Nuevo mensaje en el chat",

            "mensaje": f"Nuevo mensaje en {titulo_doc}",

            "fecha": ahora,

            "leida": False,

        }

        await request.app.state.db["notificaciones"].insert_one(notif)

        await manager.notificar(receptor, {

            "tipo": "nueva_alerta",

            "titulo": notif["titulo"],

            "mensaje": notif["mensaje"],

        })

    return {"mensaje": "Mensaje enviado exitosamente"}
```

DIAGRAMA DE FLUJO (DF) Chat en Tiempo Real por Documento

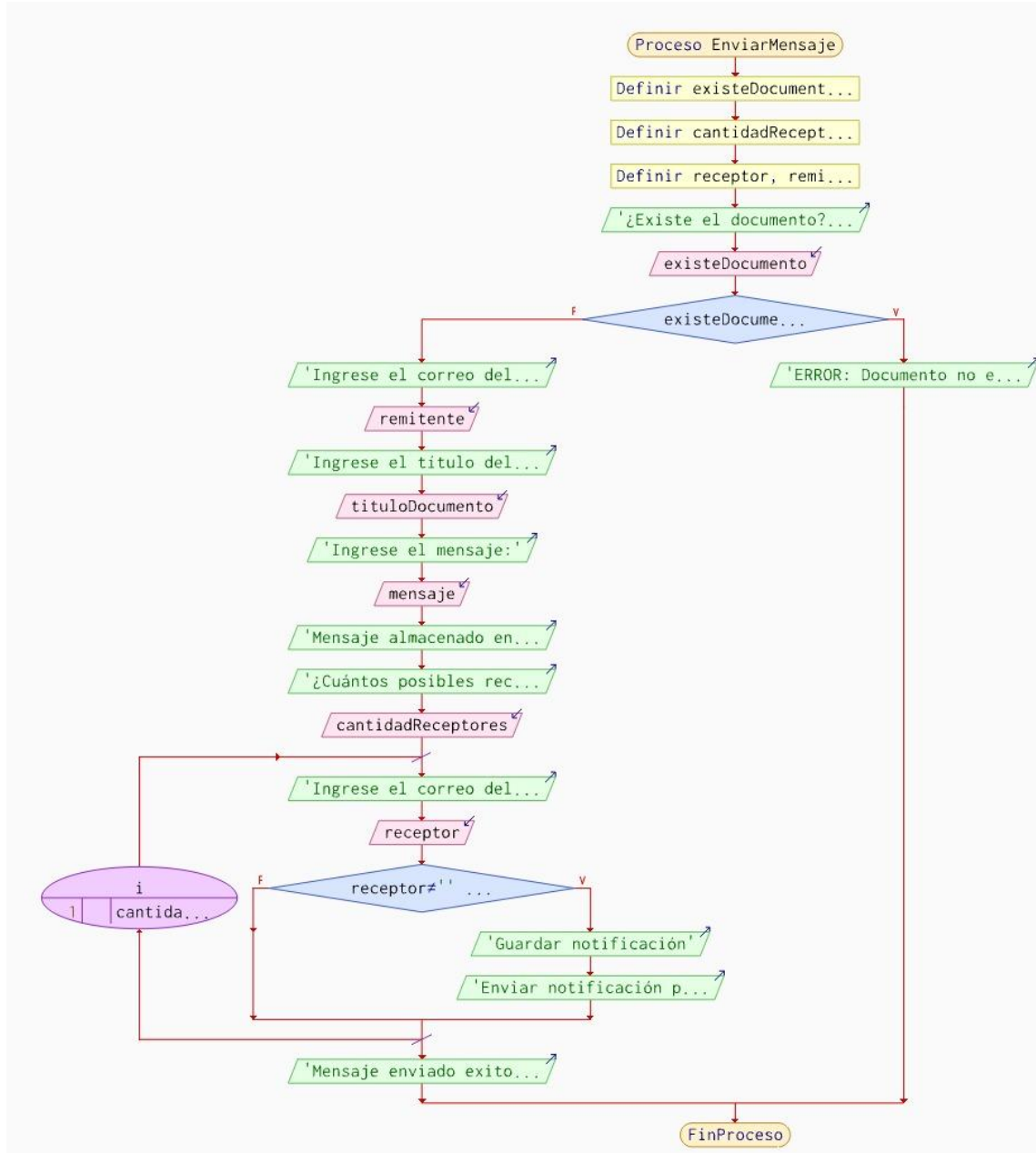
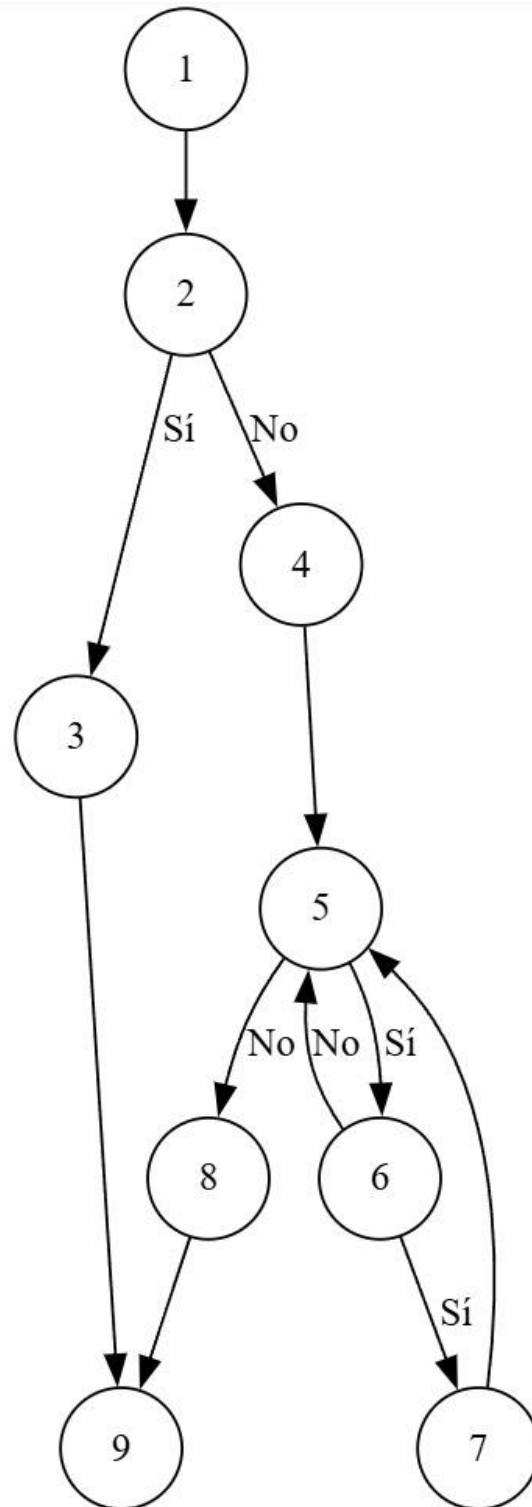


DIAGRAMA DE FLUJO (DF) Chat en Tiempo Real por Documento



IDENTIFICACIÓN DE LAS RUTAS Chat en Tiempo Real por Documento

R1: $1 \rightarrow 2 \rightarrow 3 \rightarrow 9$ (El documento no existe, entra al condicional de error, se lanza la excepción y el proceso termina)

R2: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 5 \rightarrow 8 \rightarrow 9$ (El documento existe, recorre los posibles receptores, encuentra un receptor válido diferente al remitente, guarda y envía la notificación, finaliza el bucle y termina con éxito)

R3: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 5 \rightarrow 8 \rightarrow 9$ (El documento existe, recorre los posibles receptores, pero el receptor es inválido o es igual al remitente, por lo que salta el envío, finaliza el bucle y termina con éxito)

R4: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 8 \rightarrow 9$ (El documento existe, pero la lista de posibles receptores está vacía, saltando directamente al mensaje de éxito)

COMPLEJIDAD CICLOMÁTICA

Método 1

$V(G) = \text{Nodos predicados (IF-THEN-ELSE o bucles)} + 1$ En el grafo existen tres estructuras de decisión o nodos predicado (nodos 2, 5 y 6):

Nodo 2 (Validación de si el documento no existe)

Nodo 5 (Control de iteración del bucle FOR de receptores)

Nodo 6 (Validación de si el receptor es válido y diferente al remitente) Por tanto:

$$P = 3$$

$$V(G) = P + 1$$

$$V(G) = 3 + 1$$

$$V(G) = 4$$

Método 2

$V(G) = A - N + 2$ Donde:

$N = 9$ nodos

$A = 11$ aristas

Sustituyendo:

$$V(G) = A - N + 2$$

$$V(G) = 11 - 9 + 2$$

$$V(G) = 4$$

DONDE: $P = \text{Número de nodos predicado} = 3$

$A = \text{Número de aristas} = 11$

$N = \text{Número de nodos} = 9$

Resultado

$$V(G) = 4$$

Conclusión

La complejidad ciclomática del módulo Chat en Tiempo Real por Documento es 4, por lo que se requieren cuatro casos de prueba independientes para cubrir todos los caminos posibles del código.